PKCERT AI & Software Development Internship
Task 18: Intro to PyTorch

Abdullah Amir

AI and Software Development
28 July 2026

This report covers the theory and the results. The full code and outputs live in the notebook `intro_to_pytorch.ipynb`. **PyTorch 2.13.0 (CPU build)** is used throughout. Task 17 built a perceptron, activation functions, and a 2-layer MLP from scratch in NumPy, with backpropagation verified against finite differences. This task rebuilds the same architecture on the same dataset in PyTorch, using its autograd engine and `nn.Module` API, and maps every concept back to that manual derivation.

Part A: Tensors & Basic Operations

Four ways to create a tensor, each suited to a different situation: `torch.tensor(list)` for concrete hand-written values; `torch.zeros/ ones` for placeholders and accumulators of known shape; `torch.arange` for index ranges and schedules; `torch.rand` for weight initialisation and stochastic sampling; `torch.from_numpy` for data that already lives in NumPy and needs to enter PyTorch without copying.

Indexing, slicing, reshaping. Standard NumPy-style indexing and slicing work directly on tensors. `view()` always shares the underlying memory (and requires the tensor to already be contiguous); `reshape()` behaves the same way when possible but silently falls back to copying when the current memory layout does not support the requested shape – the safer general-purpose choice, since it never raises where `view()` would. **Broadcasting** follows NumPy's rules exactly: a $(4, 1)$ tensor plus a $(1, 3)$ tensor broadcasts to $(4, 3)$ with no data copied; genuinely incompatible shapes, e.g. $(4, 2)$ and $(3, 2)$, correctly raise a `RuntimeError`.

Matrix multiplication, NumPy vs PyTorch, timed. A 1200×1200 matmul, mean of 5 runs: NumPy 32.50ms, PyTorch 45.08ms – NumPy was 1.39x faster on this CPU. Both libraries ultimately call into an optimised BLAS library for float64 CPU matmul, so this gap reflects which BLAS backend each happened to link against on this particular machine, not an inherent advantage of either library's Python layer.

`torch.Tensor` vs `numpy.ndarray`. Demonstrated, not just described: `torch.from_numpy()` and `.numpy()` both construct a view over the *same* memory buffer – mutating a tensor created with `from_numpy()` was confirmed to mutate the original NumPy array, and vice versa via `.numpy()`. `torch.tensor(existing_array)`, in contrast, always copies: mutating the resulting tensor left the original array untouched. Underneath the API, a `torch.Tensor` and an `ndarray` are the same kind of object – a contiguous memory buffer plus shape/stride/dtype metadata – which is exactly why the sharing conversions are free and the copying ones are not.

Part B: Autograd

Computational graph. A record of every differentiable operation applied to a tensor, linking each output back to the inputs and operation that produced it – what `.backward()` walks in reverse to apply the chain rule automatically. PyTorch builds this **dynamically** ("define-by-run"): no graph exists before any code runs; each operation appends a node *as it actually*

executes, which is why ordinary Python control flow works transparently inside a PyTorch model.

A scalar check. $y = x^2 + 3x + 1$ at $x = 3.0$: autograd gives $dy/dx = 9.0$; by hand, $2x + 3 = 9.0$. Match confirmed with `torch.isclose`.

Gradient accumulation. Calling `.backward()` three times on $(w - 5)^2$ without resetting `w.grad` between calls accumulated $-6, -12, -18$ – each call *adds* to the existing gradient rather than replacing it. This is why every training loop calls `optimizer.zero_grad()` before each new `loss.backward()`: without it, every update silently includes every previous step's left-over gradient (the deliberate exception is genuine gradient accumulation across mini-batches to simulate a larger batch, where the accumulation is the point).

`torch.no_grad()` and `.detach()`, proven rather than asserted. Attempting `weight -= lr * weight.grad` on a leaf tensor with `requires_grad=True`, outside `no_grad()`, was run and actually raised: `"a leaf Variable that requires grad is being used in an in-place operation"`. Wrapping the identical line in `with torch.no_grad():` succeeded. `no_grad()` is the right tool for a whole block of computation (an evaluation pass, a manual parameter update); `.detach()` is for pulling one specific tensor out of the graph mid-computation (e.g. logging a loss value) without carrying its upstream graph.

Autograd vs Task 17's manual backpropagation

The exact weights Task 17 trained and saved were loaded, and both the manual NumPy formulas and PyTorch autograd were run on the identical batch of real penguin data.

Parameter	Max abs. difference, autograd vs manual
W_1	$\approx 10^{-17}$
b_1	$\approx 10^{-17}$
W_2	$\approx 10^{-17}$
b_2	$\approx 10^{-17}$

Table 1: All four match to machine epsilon for float64 – essentially exact.

Two independent gradient checks now agree. Task 17 verified its manual backprop against finite differences ($\sim 10^{-10}$ relative error, an analytical-vs-numerical-approximation comparison). This check compares the same manual formulas against PyTorch's autograd – two *exact* analytical methods – and they agree to $\sim 10^{-17}$, machine epsilon itself. Autograd is not doing something different or more mysterious than the hand derivation; it is automating the identical calculus.

Part C: Building & Training a Simple Neural Network

Same dataset (Palmer Penguins, 342 birds, 4 features, 3 species) and architecture ($4 \to 8 \to 3$, ReLU hidden) as Task 17. A fresh `ManualMLP` (NumPy) and `MLPClassifier` (scikit-learn) were also trained in this same run, on the same split, so all three training times are measured on identical hardware rather than compared across sessions.

Loss and optimizer, justified. `nn.CrossEntropyLoss` combines `log_softmax` and negative-log-likelihood in one numerically stable call (log-sum-exp internally, avoiding the separate softmax-then-clip Task 17 needed by hand) – the direct PyTorch analogue of the manually derived softmax+cross-entropy pair. `SGD` at the same learning rate as Task 17/sklearn (0.1) is

used for the primary, fair three-way comparison; `Adam` is explored separately as the hyperparameter experiment.

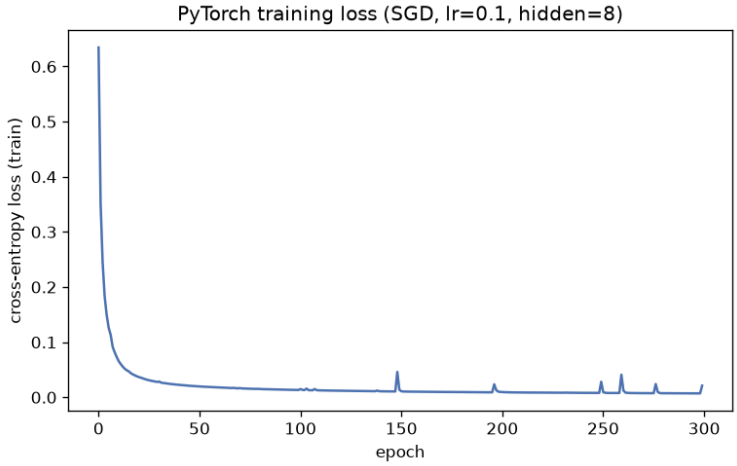

Figure 1: PyTorch training loss, SGD, 300 epochs.

Implementation	**Accuracy**	**Macro-F1**	**Fit time**
ManualMLP (NumPy, Task 17)	0.9855	0.9877	**0.110s**
sklearn MLPClassifier	**1.0000**	**1.0000**	**0.109s**
PyTorch nn.Module (SGD)	**1.0000**	**1.0000**	2.510s

Table 2: Same 300-epoch, batch-16 schedule, same split, same CPU.

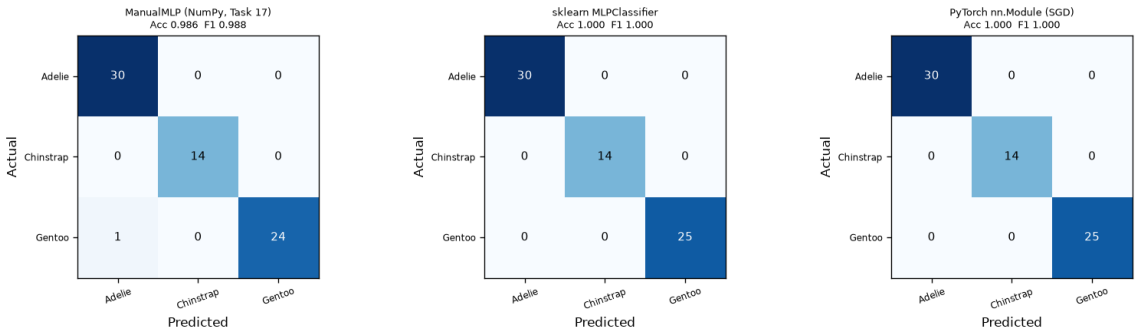

Figure 2: All three confusion matrices; PyTorch and sklearn are both perfect on this split.

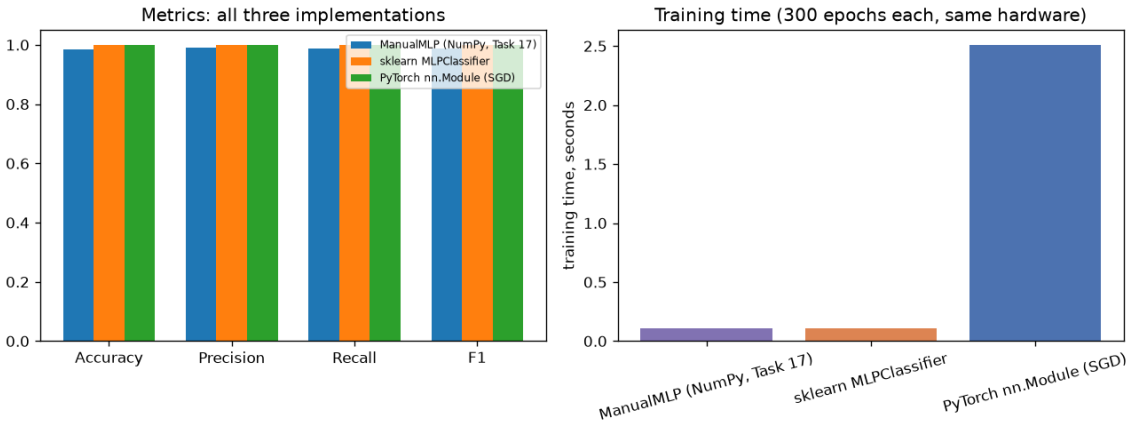

Figure 3: Left: metrics across all three. Right: training time – PyTorch is the outlier.

PyTorch matched sklearn on accuracy but took roughly 20x longer to train than either the NumPy or scikit-learn model, despite all three running the identical schedule on the same CPU. This is not a criticism of PyTorch – it is the expected result of per-operation overhead (building an autograd graph node, Python/C++ dispatch, gradient bookkeeping for every one of the roughly 5,400 mini-batch steps) dominating when the actual arithmetic per step is tiny (a 4×8 and an 8×3 matrix multiply). Frameworks like PyTorch earn their keep on large models, large batches and GPU execution, where that per-step overhead becomes negligible next to the computation itself; on a network this small, the overhead *is* most of the cost.

Hyperparameter experiment: optimizer choice

Optimizer	Final training loss	Test accuracy
SGD (lr=0.1)	0.0212	**1.0000**
Adam (lr=0.1)	0.0019	0.9855
Adam (lr=0.01)	**0.0012**	**1.0000**

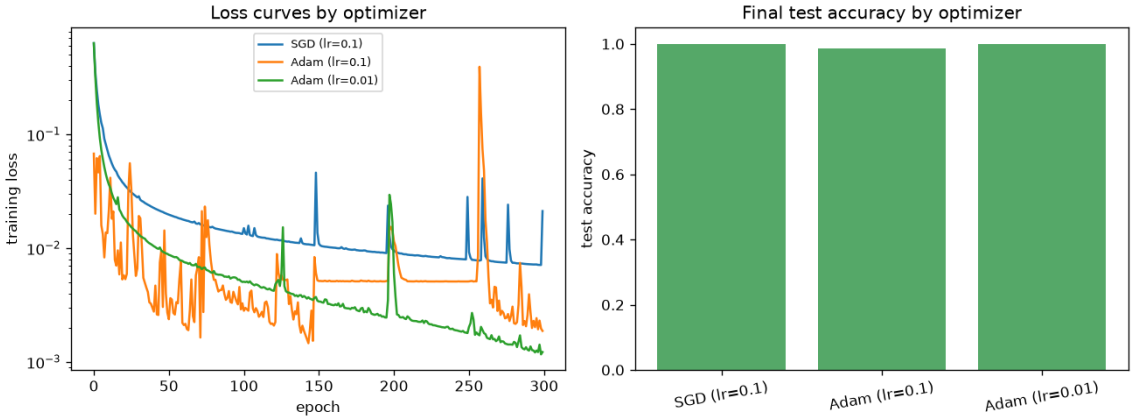

Figure 4: Left: loss curves (log scale) by optimizer. Right: final test accuracy.

Adam at the same learning rate as SGD (0.1) performed *worse* (98.6% vs SGD's 100%) and its loss curve is visibly noisier – Adam's adaptive per-parameter scaling makes its *effective* step size at a given nominal learning rate larger than plain SGD's, and 0.1 is well

outside Adam's typical comfortable range. Dropping to `lr=0.01` fixed both symptoms: the smoothest curve of the three and the lowest final loss, back to 100% test accuracy. The lesson is specific and checkable, not a general claim that either optimizer is better: an optimizer swap is not a drop-in change at a fixed learning rate.

Part D: Analysis & Documentation

Pipeline summary. Architecture $4 \to 8 \to 3$ (identical to Task 17): `nn.Linear(4,8)` $\to$ ReLU $\to$ `nn.Linear(8,3)`, raw logits out. Loss: `nn.CrossEntropyLoss`. Optimizer for the primary comparison: SGD(lr=0.1), matching the manual and sklearn baselines exactly; Adam explored separately. 300 epochs, batch size 16 – identical to Task 17 throughout. The trained model's `state_dict` was saved with `torch.save` and reloading it with `load_state_dict` was verified to reproduce identical test predictions.

Two concrete differences observed, Task 17 (NumPy) vs this PyTorch implementation.

1. *Training speed, in the opposite direction most people expect.* PyTorch took ~20x longer than the hand-written NumPy version on this tiny network, because per-step framework overhead dominates when the actual matrix operations are this small. The lesson generalises: a framework's advantage is in what it makes *possible* (deeper networks, GPU execution, automatic differentiation through arbitrary architectures) and in engineering time, not necessarily wall-clock speed on a toy-sized problem.

2. *Code complexity and the shape of the risk.* The manual implementation is roughly 60 lines of NumPy that must be re-derived by hand for any architecture change, with the entire correctness burden on getting the calculus and matrix orientations right (Task 17 needed an explicit finite-difference check to trust it). `loss.backward()` replaces the entire hand-written `backward()` method, and autograd is correct by construction for any differentiable graph, so the risk shifts away from calculus bugs and toward framework-specific mistakes: dtype/shape mismatches, forgetting `zero_grad()`, or an in-place operation on a leaf tensor (the gotcha below).

What autograd abstracts away, and why Task 17 still mattered. Autograd abstracts away the entire derivation this pipeline depends on: it is never told that $\partial L/\partial Z_2 = A_2 - Y$, or which axis a bias gradient sums over, or which matrix to transpose in $dW_1 = X^\top dZ_1$ – it derives all of that automatically from the chain rule applied to whatever `forward()` happens to run. That is exactly why Task 17 remains valuable even though this task never needs its formulas directly again: writing and finite-difference-verifying the derivation by hand is what makes it possible to read a loss curve, a vanishing gradient, or a training instability and know which piece of the underlying mathematics is responsible, rather than treating `loss.backward()` as a button that makes numbers go down. Part B's $\sim 10^{-17}$ agreement between autograd and the Task 17 formulas is the direct proof that the two compute the same thing.

A limitation/gotcha, demonstrated live. Calling `weight -= lr * weight.grad` on a leaf tensor with `requires_grad=True`, outside `torch.no_grad()`, raises immediately – PyTorch forbids in-place updates to such tensors because they would silently corrupt values other operations may still need during backpropagation. The fix, and the reason `optimizer.step()` wraps itself in `no_grad()` internally, is to explicitly mark that block of code as untracked by autograd – caught and printed directly in Part B rather than only described in prose.

Conclusion. The thread connecting this task to Task 17 is verification, not just translation. Every claim here was checked against an independent source: PyTorch's own tensor semantics were demonstrated with actual shared-memory mutations and an actual caught `RuntimeError`,

not just described; and, most importantly, PyTorch's autograd gradients were checked against Task 17's hand-derived formulas on identical weights and data, agreeing to machine precision. The result is a PyTorch implementation that reaches the same accuracy as the framework baseline it is meant to reproduce, at a real and now-quantified cost in training time for a network this small – evidence that adopting a production framework is a genuine trade-off, not a strict upgrade, at this scale.

The notebook and README are on GitHub: https://github.com/AbdullahAmir06/ai-internship-abdullahamir